

BizSim — Engineering Report

Project: BizSim, Business Continuity Intelligence — SMVIT ValueHack 2026

Branch: p1-frontend-readme-report

Iteration: frontend rebuild, README rewrite, verification

Date of this run: 3 October 2026

Author: Person 1 (engine + frontend + documentation)

1. Executive summary

BizSim is a deterministic 30-day cascade simulator: it takes one disruption (late supplier, late customer, cost spike) and shows the chain it sets off — inventory shortage → delayed orders → customer payment delay → cash-flow gap → failed supplier payment — with a plain-language explanation and a ranked set of actions the owner could take. Its stated philosophy is **estimates, not predictions; the tool suggests, the owner decides**.

This iteration added no engine features. It (a) rebuilt the frontend so it actually displays what the engine already returns, (b) replaced the README with a verified document, and (c) produced this report. ***No backend source file was changed*** — the backend was read-only, as briefed.

Headline results reproduced end to end in the browser for the demo scenario (Supplier A 14 days late): stock-out on **day 10**, lost walk-in sales

■**1,77,600**, cash below zero on **day 23**, lowest cash ■**-1,71,600** on day 25, **2 failed payments** (Supplier A day 23, Supplier C day 25). The engine suggests **asking Supplier A for an extension** (Medium risk, no cash gap, needs counterparty agreement); the switch-to-new-distributor option is ***not affordable with a ■29,225*** shortfall.

Verification: backend **50 tests pass in 2.58 s**; frontend `npm run lint`, `npm run tsc --noEmit` and `npm run build` all pass; all **12 endpoints** smoke-tested (HTTP 200) and both documented error codes reproduced. Twelve UI checks were executed in a real browser against the live backend and are tabulated in §6 — including three defects that were found and fixed during verification (see §9). Everything the UI shows is labelled as an estimate and comes from the API; no business figure is calculated in the frontend.

2. Scope and method

In scope: replacing the frontend screens' hard-coded demo numbers and dead placeholders with real API data; adding the missing screens (price what-if); rewriting the README; writing this report; verification and evidence capture.

Out of scope: the simulation engine itself; `signals/*`, `explain/report.py`,

forecast.py, anomaly.py, bills.py, gstin.py; the /api/attention stub (Person 2); real vendor registry integration (Person 4); CSV business loading; automated UI test suites.

Method, in phase order:

1. **Discovery** — read docs/api_contract_engine.md, engine_decisions.md, engine_audit.md and the git history; ran `pytest -q`; started the backend and captured **real, trimmed responses** for every endpoint into docs/samples/; read every frontend page; wrote docs/frontend_gap_list.md (a *backend returns X, the UI does not show it* list, 30 items).
2. **Frontend rebuild** — typed API client and shared UI primitives first, then the Fire Drill, then the remaining screens, each verified against the live backend in a browser.
3. **README rewrite** — every feature status justified from code, every command run, file tree generated from the real filesystem, versions read from the installed environment (because requirements.txt is unpinned).
4. **Verification** — fresh pytest, lint, type check, build, endpoint smoke test, error-format check, secret scan, and a manual browser walk of the whole demo path at desktop and 360 px widths.

Evidence discipline: every number in this report comes from a command run or an API capture in this session. Items that were not executed are marked *Not executed* rather than assumed.

3. Backend work completed (previous iteration — context for this one)

The engine was built in the prior iteration and is summarised here from docs/engine_audit.md, docs/engine_decisions.md and the code. ****Nothing in this section was modified during this iteration.****

Step	What changed	Files
Audit	Old demo constants inventoried; full responses frozen as fixtures so lega	tests/gold
Data-d	Business, Product, Supplier, Customer, Shock, Plan replace constants bake	engine/mod
Simula	Day loop (receive stock → deliver orders → sell → collect → pay → record	engine/sim
Scenar	supplier_delay, customer_delay, cost_spike, demand_shock; target resolves	engine/cas
Explan	One sentence per summary metric, each containing its own formatted value,	explain/re
Assump	Assumption list with source (demo_data/assumed/user) and editability; low	explain/re
Action	Six actions as full cascade results plus delta_vs_do_nothing, tradeoff, a	engine/act
Pricin	Price what-if with elasticity bands, 21-point curve, break-even demand dr	engine/pri
Analyt	Supplier dependency shares with concentration bands, stock cover with saf	engine/ana
Error	One EngineError → {code, message}, status 422/404, seven codes	engine/err
Seed	Demo business isolated to one dict	seed.py

Additive API surface: /api/pricing/whatif was added (new router); other endpoints gained response fields only. Legacy fields (stockout, lost_rev, first_negative_day, min_cash, min_day, failed, delivered, end_cash, cash_timeline, stock_timeline) are unchanged and still asserted by the golden tests, which is why the older screens kept working while the new ones were built.

Backend changes made during this iteration: none. The only repository-level

change outside frontend/ and docs/ is a new root .gitignore (§9, item 3).

4. Frontend work completed

4.1 Foundations (new files)

File	What it does
lib/ty	TypeScript types for every response in the contract (legacy fields, summary, cascade_c
lib/ap	Rewritten typed client. One ApiError that parses the backend {code, message}, maps cod
lib/fo	Indian-grouped rupee formatter (■1,77,600, negatives ■-1,71,600), day/count/percent fo
compon	Loading skeleton, empty state, failed state with Retry and an offline hint that names
compon	Estimate badge, risk badge (text + colour), safe/watch/critical chip, concentration ch
.env.l	Documents the one frontend env var

4.2 Fire Drill (app/fire-drill/page.tsx + components/fire-drill/*)

- **ScenarioControls** — type selector (supplier delay / customer delay / cost spike), target field (hidden for a cost spike), magnitude slider + number input with per-type units, 0–50 demand band, and the plain-words box that calls POST /api/scenario/parse and fills the controls (owner can edit). The parser's actual behaviour (regex stub, supplier delays only) is stated in the UI when it returns something the screen cannot use.
- **Auto-run** — 300 ms debounce on any control change; every run aborts the previous request (AbortController), so stale results cannot overwrite fresh ones.
- **CascadeMap** — React Flow map drawn from cascade_chain: six nodes with title, day, detail and status, staggered Framer Motion reveal that is disabled under prefers-reduced-motion, click-to-select (marks the step's day on the cash chart and clears on pane click), and a two-column layout below 640 px so a 360 px phone has no horizontal scroll.
- **TimelinesChart** — cash lines (this scenario, the do_nothing baseline from the action comparison, and the suggested action), zero line, first-negative-day marker, failed-payment dots at the exact day/cash point, selected-step and highlighted-event day lines, plus a per-product stock chart with the stock-out marker. Both charts carry role="img" and descriptive aria-labels.
- **WhyPanel** — the engine's explanations as plain sentences with metric chips; clicking one highlights the events it names (and their days on the chart). Beside it, the event timeline with day, severity chip, type, entity, values and a highlight ring on selected events.
- **AssumptionsBox** — always visible, every assumption with its source tag and an “editable” marker.
- **RangesPanel** — min/base/max per metric from ranges.metrics plus the engine's range_note.
- **ActionsCompare** — a card per action with stock-out/lowest cash/end cash/failed-payment counts, deltas versus doing nothing, the tradeoff sentence,

risk badge with expandable per-factor reasons, affordability (or the shortfall amount), the counterparty-agreement tag, per-action demand range, the engine's suggestion, and for `switch_supplier` the vendor band, score, reasons, exposure and an "if the vendor vanishes" toggle showing that run's numbers, plus a link to the vendor check screen.

- **No business figures are computed in the UI** — only formatting and drawing.

4.3 Other screens

- **Dashboard** — KPIs from `/api/business`, supplier dependency bars with concentration chips and both value shares, stock cover with status chips and days-to-stockout, the engine's `attention_inputs` with their evidence text, and the `/api/attention` list rendered separately and **labelled as a demo stub**. The previous hard-coded KPIs (12 days, ■70,000, 75%) and the fabricated supplier bar chart are gone.
- **Pricing (new, /pricing)** — product picker, elasticity case (low/base/high, labelled as an assumption), slider `-20...+20` that reads the precomputed curve so it never refetches, profit low/base/high, break-even demand drop, the engine sentence, and the assumption list.
- **Report** — now built from `POST /api/actions/compare` for the chosen delay: the do-nothing chain of consequences, the engine's suggested action with its reason and figures, and a working PDF download from `GET /api/report`.
- **Vendor check** — unchanged mock check, but the dead "use this vendor" button now deep-links to `/fire-drill?target=<name>`, which prefills the Fire Drill target. Errors now surface as a friendly failed state.
- **Bill scan** — upload now calls the real (stub) `/api/bill/scan` and the result is labelled **demo stub output**; the page no longer implies extraction works.
- **TopNav** — added the Price What-If link; the nav row scrolls horizontally on small screens instead of disappearing, and the active-link underline was fixed (it was escaping the scroll container and widening the page).
- **About** — engine module list corrected to the real files, including that the scenario parser is a regex and no LLM call exists.

4.4 Accessibility and responsive behaviour

Keyboard-reachable controls with visible focus rings, `aria-pressed` on the scenario/elasticity toggles, `role="img"` + labels on charts and the cascade map, severity and status always shown as text beside colour, and a two-column cascade layout below 640 px. Verified at 1280×900 and 360×740 .

4.5 What remains in the frontend

No automated UI tests; the bill-scan and report flows depend on backend stubs; `next.config.mjs` still skips type/lint checks during `next build`, so

`npm run tsc --noEmit` and `npm run lint` must be run separately (both clean).

5. API contract summary

Twelve endpoints are live (11 in `main.py`, `POST /api/pricing/whatif` in `routers/engine.py`): `GET /health`, `GET /api/business`, `GET /api/attention`, `GET /api/forecast`, `GET /api/anomalies`, `GET /api/report`, `POST /api/simulate/cascade`, `POST /api/actions/compare`, `POST /api/pricing/whatif`, `POST /api/scenario/parse`, `POST /api/vendor/check`, `POST /api/bill/scan`. Full request/response shapes, real examples and the error format are in `docs/api_contract_engine.md` and `README.md`; raw captures are in `docs/samples/`.

Backward compatibility. The cascade response keeps every legacy field (`stockout`, `lost_rev`, `first_negative_day`, `min_cash`, `min_day`, `failed`, `delivered`, `end_cash`, `cash_timeline`, `stock_timeline`) alongside the new blocks, and `tests/test_golden.py` asserts the frozen fixtures still match. The frontend rewrite happened to consume only the new blocks, but it was free to do so *because* the legacy contract was frozen first.

Error contract. One shape for all engine failures — `{"code", "message"}`, status 422 for invalid input and 404 for unknown entities. Codes: `INVALID_SHOCK`, `UNKNOWN_SUPPLIER`, `UNKNOWN_CUSTOMER`, `UNKNOWN_PRODUCT`, `MAGNITUDE_OUT_OF_RANGE`, `ELASTICITY_OUT_OF_RANGE`, `INVALID_INPUT`. Verified live in this session:

```
POST /api/simulate/cascade {"scenario":{"type":"nonsense"...}}
→ {"code":"INVALID_SHOCK","message":"Unknown scenario type 'nonsense'."}
POST /api/pricing/whatif {"elasticity":-9}
→ {"code":"ELASTICITY_OUT_OF_RANGE","message":"Elasticity must be between -3.0 and 0.0."}
```

6. Test results (real)

6.1 Backend suite

Command: `cd backend && pytest -q` (also make test).

```
50 passed, 20 warnings in 2.58s
```

50 tests collected; 0 failed, 0 skipped, 0 errors. The 20 warnings are `SQLModel.session.query()` deprecation notices from `app/engine/adapters.py`, unrelated to the results.

6.2 Representative backend test cases

Every row below was executed in the run above. “Actual” is what the assertion checked against; the arithmetic for the oracle rows is written out in `backend/tests/test_oracle.py`. Money in rupees.

ID	Scenario	Input	Expected	Actual	Result
B01	Oracle — bas	Plan()	stock-out 30, lost 14800, first-	matches	Pass

B02	Oracle – 14-	supplier_delay A	10, 177600, first-negative 23, l	matches	Pass
B03	Oracle – ext	delay 14 + invoic	10, 177600, first-negative none,	matches	Pass
B04	Oracle – dis	delay 14 + early_	10, 177600, first-negative 23, l	matches	Pass
B05	Oracle – ext	delay 14 + extens	10, 177600, none, 62550 @ d1, no	matches	Pass
B06	Oracle – new	delay 14 + switch	stock-out 30, lost 14800, first-	matches	Pass
B07	Oracle – ven	delay 14 + switch	10, 310800, first-negative 3, lo	matches	Pass
B08	Oracle – cos	cost_spike A 15	30, 14800, none, 38750 @ d25, no	matches	Pass
B09	Oracle – cos	cost_spike A 30	30, 14800, first-negative 25, -2	matches	Pass
B10	Oracle – lat	customer_delay Ve	30, 14800, none, 6000 @ d25, non	matches	Pass
B11	Headline num	delay 14	stock-out 10, first-negative 23,	matches	Pass
B12	Headline ove	POST /api/simulat	200 with stockout 10, first_nega	matches	Pass
B13	Cascade chai	delay 14	steps in order with days 8, 10,	matches	Pass
B14	Property – s	delays 0,2,5,10,1	stock-out day never gets later a	sequence non-in	Pass
B15	Property – l	same delays	lost sales never decrease	sequence non-de	Pass
B16	Property – d	delay 14 run twic	identical summary, timeline and	identical	Pass
B17	Property – r	delay 14 with ran	base run equals summary; min ≤ b	matches	Pass
B18	Explanation	delay 14	every summary key has an explana	complete	Pass
B19	Explanation	baseline, delay 1	each sentence contains its own f	matches	Pass
B20	Rupee format	0, 999, 1000, 100	■0 ... ■-1,71,600	matches	Pass
B21	Validation –	scenario.type "ea	422 {code: INVALID_SHOCK}, keys	matches	Pass
B22	Validation –	target „nope“	404 UNKNOWN_SUPPLIER	matches	Pass
B23	Validation –	elasticity -5 (en	ELASTICITY_OUT_OF_RANGE (422 ove	matches	Pass
B24	Pricing arit	fans +10%, elasti	price 2090, margin 500→690, mult	matches	Pass
B25	Pricing curv	fans +10%	21 points, -20...+20, each with lo	matches	Pass
B26	Pricing assu	fans +10%	price_elasticity has source: "as	matches	Pass
B27	Timing – act	delay 14, warm	under 300 ms	24.6–30.1 ms (m	Pass
B28	Timing – dem	delay 14	under 300 ms	1.8–3.3 ms (med	Pass
B29	Golden legac	baseline, delay 1	legacy names/types/values unchan	matches	Pass
B30	Analytics	/api/business	dependency shares, stock cover,	matches	Pass

Additional measured timing (not an assertion): price_whatif for fans +10% ran

34.7–55.4 ms (median 35.9 ms) over 5 warm runs, well inside the same budget.

6.3 UI test cases

Executed in the in-app Chromium browser against the **live backend** on

<http://localhost:3100>, unless marked otherwise. “Observed” is what was actually

seen, quoted from the page or the network log.

ID	Test case	Executed?	Observed result
U01	Load the demo	Executed	Summary strip showed Stock-out Day 10, lost sales ■1,77,6
U02	Cascade map s	Executed	Six nodes in order – Supplier delay d8, Inventory shortag
U03	Explanation h	Executed	Clicking “First stock-out day” moved the panel into highl
U04	Switch scenar	Executed	Controls changed to “Cost increase”, the supplier field d
U05	Change the de	Executed	Slider moved to ±45% with real key input; the engine's ra
U06	Action compar	Executed	“Switch to the new distributor” card: stock-out Day 30 (+
U07	The suggestio	Executed	“ENGINE SUGGESTS” badge sat on Ask Supplier A for an exte
U08	Price slider	Executed	Slider set to -6% → profit showed ■44,776 immediately; th
U09	360 px width	Executed	After fixing two overflow causes, documentElement.scrollW
U10	Backend offli	Executed	With the backend killed, the dashboard showed “The dashbo
U11	Vendor check	Executed	“New Distributor” returned score 32, red band, cheque-bou
U12	Report page a	Executed	Report rendered the do-nothing consequences and the engin
U13	Bill-scan upl	Not executed in	The upload control is wired to the stub endpoint and labe
U14	Automated UI	Not executed	No suite exists and no browser automation package is inst
U15	Keyboard-only	Not executed	Focus rings, aria-pressed, role="img" labels and text-bes
U16	Reduced-motio	Not executed	The code uses useReducedMotion() to disable the cascade r
U17	Loading / emp	Partially execu	Loading skeletons and the failed state with Retry were se

6.4 Build, lint and type checks

```

npx next lint          → ✓ No ESLint warnings or errors
npx tsc --noEmit       → ✓ clean (no output; zero type errors)
npm run build          → ✓ Compiled successfully, 11 static pages generated
                        /fire-drill 101 kB (304 kB first load), /pricing 4.99 kB,
                        /dashboard 4.74 kB, /report 4.8 kB, /bill-scan 3.69 kB,
                        /vendor-check 3.53 kB, /about 1.58 kB

```

Note: next build alone does **not** type-check in this repo because

next.config.mjs sets typescript.ignoreBuildErrors and

eslint.ignoreDuringBuilds. npx tsc --noEmit was therefore run separately — it

found 9 type errors on the first pass (all in the new files, unknown used

where a `ReactNode` was required, a stale prop and a tooltip formatter signature), and all 9 were fixed before this report.

7. Sample outputs

Trimmed real responses are saved in `docs/samples/` (captured live by `docs/samples/capture_samples.py`; long arrays are truncated with an explicit marker, values are unedited):

File	Endpoint / case
business.json	GET /api/business with
cascade_delay14.json	14-day Supplier A dela
cascade_cost_spike30.json	30% cost spike
cascade_customer_delay7.json	Verma pays 7 days late
actions_delay14.json	Six-action comparison,
pricing_whatif_fans_plus10.json	fans +10%, 21-point cu
scenario_parse.json, vendor_check.json, bill_scan_stub.json, attentio	The remaining endpoint

Headline excerpt from `docs/samples/cascade_delay14.json`:

```
{"stockout": 10, "lost_rev": 177600.0, "first_negative_day": 23,
"min_cash": -171600.0, "min_day": 25, "end_cash": -47600.0,
"failed": [{"name": "Supplier A invoice", "day": 23},
{"name": "Supplier C invoice", "day": 25}],
"summary": {"stockout_day": 10, "lost_sales_inr": 177600.0,
"lost_sales_all_products_inr": 203100.0, "first_negative_day": 23,
"lowest_cash_inr": -171600.0, "lowest_cash_day": 25, "end_cash_inr": -47600.0,
"failed_payments_count": 2, "order_delay_days": 10, "customer_payment_day": 29}}
```

Action excerpt from `docs/samples/actions_delay14.json`:

```
{"ask_extension": {"suggested": true, "affordable": true,
"requires_counterparty_agreement": true, "risk": {"level": "Medium", "score": 2},
"delta_vs_do_nothing": {"lowest_cash_inr": {"base": -171600.0, "action": 62550.0, "delta": 234150.0}}},
"switch_supplier": {"affordable": false, "shortfall_inr": 29225.0,
"stockout": null, "summary": {"stockout_day": 30},
"vendor_risk": {"score": -1, "band": "unknown"}}
```

Visual artifact: `docs/screenshots/demo-walkthrough.webm` — a recording of the Fire Drill loading, an explanation click highlighting its events, page scrolling, and navigation to the pricing screen. **No PNG screenshots exist**: the browser tool used for verification renders captures inline only and writes no files, so none were fabricated. The executed UI checks in §6.3 are the substitute evidence.

8. Metrics

Code delta (`git diff --stat`):

Range	Meaning	Files	Insertions	Deletions
35b50b4..HEAD	this iteration (Phase A-D)	41	8,900	336
4bd0ca4..HEAD	the whole feature effort, engine included	99	21,504	481

The iteration's insertions are dominated by captured evidence: `docs/samples/*.json` (~5,700 lines) and the binary demo recording. Source changes this iteration: 26 frontend files plus `README.md`, `docs/frontend_gap_list.md` and `.gitignore`.

Scale

Measure	Value
Backend tests	50 (all passing)
Backend endpoint	12

Frontend routes	8 (/ , /dashboard, /fire-drill, /pricing, /vendor-check, /bill-scan, /report,
New frontend fil	12 (3 lib, 2 UI, 7 fire-drill components) + the pricing page
Branch commits	5 before this report (Phase A, foundations, Fire Drill, other screens, READM
compare_actions	24.6-30.1 ms (median 26.6)
compute_ranges	1.8-3.3 ms (median 1.9)
price_whatif	34.7-55.4 ms (median 35.9)
pytest wall time	2.58 s
Frontend build	11 pages, /fire-drill first load 304 kB

9. Assumptions, limitations, risks, and defects found

9.1 Simulation assumptions (from docs/engine_decisions.md)

Headline stock-out and lost-sales metrics are scoped to the disrupted supplier's products (all products are reported separately); "lowest cash" is the minimum of the daily timeline, not the opening balance; a failed payment is recorded **and** still deducted; invoice due dates do not move with late delivery; an early-payment discount moves payment to the delivery day; the alternative vendor charges 15% less with a 50% advance on day 3, delivery day 6 and balance 15 days later; the final cascade step reports the **last** failed payment; risk is a heuristic rubric, not a probability; price elasticity is an assumption; demand is flat at recent averages with a $\pm$ band shown instead.

9.2 Limitations

- **Stubs and mocks:** /api/attention (hard-coded list), /api/vendor/check (name-string rule), /api/scenario/parse (regex, supplier delays only), /api/bill/scale (fixed sample) and GET /api/report (a valid PDF whose text is hard-coded — it still recommends "Combined", which **contradicts** the engine's actual suggestion of the extension).
- **Not implemented:** forecast, anomaly detection, GSTIN checksum, CSV business loading, real invoice parsing (signals/gstin.py, signals/bills.py, engine/forecast.py, engine/anomaly.py, engine/cash.py are empty files).
- **Action comparison is supplier-delay only** — the endpoint accepts delay and nothing else; the UI says so when another scenario type is selected rather than showing a meaningless comparison.
- **No per-day demand band on the chart** — ranges returns metric summaries only, so a shaded low/high band would require the frontend to compute business figures, which the brief forbids; the ranges panel shows min/base/max and the engine's range_note instead.
- **Frontend build config** skips lint and type checks (next.config.mjs), so those must be run separately (they are clean today).
- **No automated UI tests**, and requirements.txt is unpinned (installed versions are recorded in the README instead).
- **Repo hygiene:** backend/bizsim.db and __pycache__ files are tracked and the repo had no .gitignore. One was added for future artifacts; the already

tracked files were deliberately left in place so teammates' branches are unaffected.

- The frontend labels every figure as an estimate and marks the demo data as synthetic; it does not attempt a confidence interval beyond the demand band.

9.3 Defects found and fixed during this iteration

1. **Horizontal overflow at 360 px** (page scrollWidth 552 vs 360): the nav's active-link underline, absolutely positioned inside a sticky header, escaped the nav's scroll container. Fixed with a relative link and an `left-0` underline; the nav row scrolls horizontally on small screens instead of being hidden entirely.
2. **Cascade map could not fit a phone** at `minZoom 0.35`. Added a two-column node layout below 640 px (`minZoom 0.2`) and verified `scrollWidth = 343 ≤ 360`.
3. **Nine TypeScript errors** in the new code surfaced only when running `npx tsc --noEmit`, because `next build` skips type checking in this repo: unknown values rendered as React children, a stale `loading` prop on the actions component, a missing key in a `Record<ScenarioType, ...>` map, and a Recharts tooltip formatter signature. All fixed; `tsc` is now clean.
4. **Hard-coded UI figures removed**: the dashboard's "12 days / ■70,000 / 75%" tiles and its fabricated supplier bar chart, the Fire Drill's invented baseline line (`50000 - i*3000`), and the report page's invented "minimum cash ■12,000" story. All now come from the API.
5. **Wrong "best" action** — the Fire Drill previously hard-coded "✓ BEST" on the combined action while the engine suggests the extension. The badge now follows suggested.
6. **Operational**: running `next build` while the dev server is live corrupts the dev server's `.next` chunks (404s, blank pages). The dev server was restarted on a cleared `.next`. This is a tooling quirk, not a code defect.

9.4 Risks

- The demo's credibility rests on synthetic data; the README and the UI say so.
- The report PDF contradicting the engine is the most likely thing a judge will notice — it is documented here and is roadmap item 1.
- Without automated UI tests, regressions in the Fire Drill are only caught by manual walks.

10. Next steps (prioritised)

1. Generate `explain/report.py` from live engine output (headline, cascade, suggested action, risk factors, assumptions) so the PDF agrees with the UI.
2. Make `POST /api/actions/compare` scenario-aware (`accept scenario`) so the

action comparison works for cost spikes and customer delays.

3. Wire `get_vendor_risk()` to a real check so `switch_supplier` can show a band, exposure and the vendor-vanishes result from real data.

4. Return per-run timelines in `ranges.runs[]` so the cash chart can shade the demand band.

5. Add a Playwright smoke test for the demo path and capture screenshots in CI.

6. Add CSV business loading with `{code, message}` validation and an onboarding screen.

7. Turn off `typescript.ignoreBuildErrors` / `eslint.ignoreDuringBuilds` and un-pin `requirements.txt` (or freeze it).

8. Implement real bill scanning and the GSTIN checksum.

11. Appendix

11.1 How to reproduce

```
# Backend
cd backend && pip install -r requirements.txt
python -m pytest -q # 50 passed
python -m uvicorn app.main:app --port 8000

# Frontend
cd frontend && npm install
npm run lint && npx tsc --noEmit && npm run build
npm run dev # then open http://localhost:3000/fire-drill

# Re-capture the API evidence (backend running)
PYTHONIOENCODING=utf-8 python docs/samples/capture_samples.py
```

11.2 Environment

Windows 11, Git Bash. Python 3.12.5 (fastapi 0.136.1, pydantic 2.13.3, sqlmodel 0.0.38, reportlab 5.0.1, pytest 7.4.3). Node v24.11.0, npm 11.6.1 (next 14.2.35, react 18, recharts 3.10.1, @xyflow/react 12.12.0, framer-motion 13.5.0, tailwindcss 3.4.1). Backend on :8000, frontend dev server on :3100 during verification (port 3000 was occupied).

11.3 Files created or changed in this iteration

.gitignore	(new)
README.md	(rewritten)
docs/BizSim_Engineering_Report.md	(new)
docs/frontend_gap_list.md	(new)
docs/samples/	(new: 14 captures + report.pdf + capture_samples.py)
docs/screenshots/demo-walkthrough.webm	(new)
frontend/.env.local.example	(new)
frontend/lib/api.ts	(rewritten)
frontend/lib/types.ts	(new)
frontend/lib/format.ts	(new)
frontend/components/ui/states.tsx	(new)
frontend/components/ui/badges.tsx	(new)
frontend/components/fire-drill/ScenarioControls.tsx	(new)
frontend/components/fire-drill/CascadeMap.tsx	(new)
frontend/components/fire-drill/TimelinesChart.tsx	(new)
frontend/components/fire-drill/WhyPanel.tsx	(new)
frontend/components/fire-drill/AssumptionsBox.tsx	(new)
frontend/components/fire-drill/RangesPanel.tsx	(new)
frontend/components/fire-drill/ActionsCompare.tsx	(new)
frontend/app/pricing/page.tsx	(new)
frontend/app/fire-drill/page.tsx	(rebuilt)
frontend/app/dashboard/page.tsx	(rebuilt)
frontend/app/report/page.tsx	(rebuilt)

frontend/app/bill-scan/page.tsx
frontend/app/vendor-check/page.tsx
frontend/app/about/page.tsx
frontend/components/nav/TopNav.tsx

(rebuilt)
(error state + deep link)
(module list corrected)
(pricing link, mobile row, underline fix)

No backend source file was changed in this iteration.